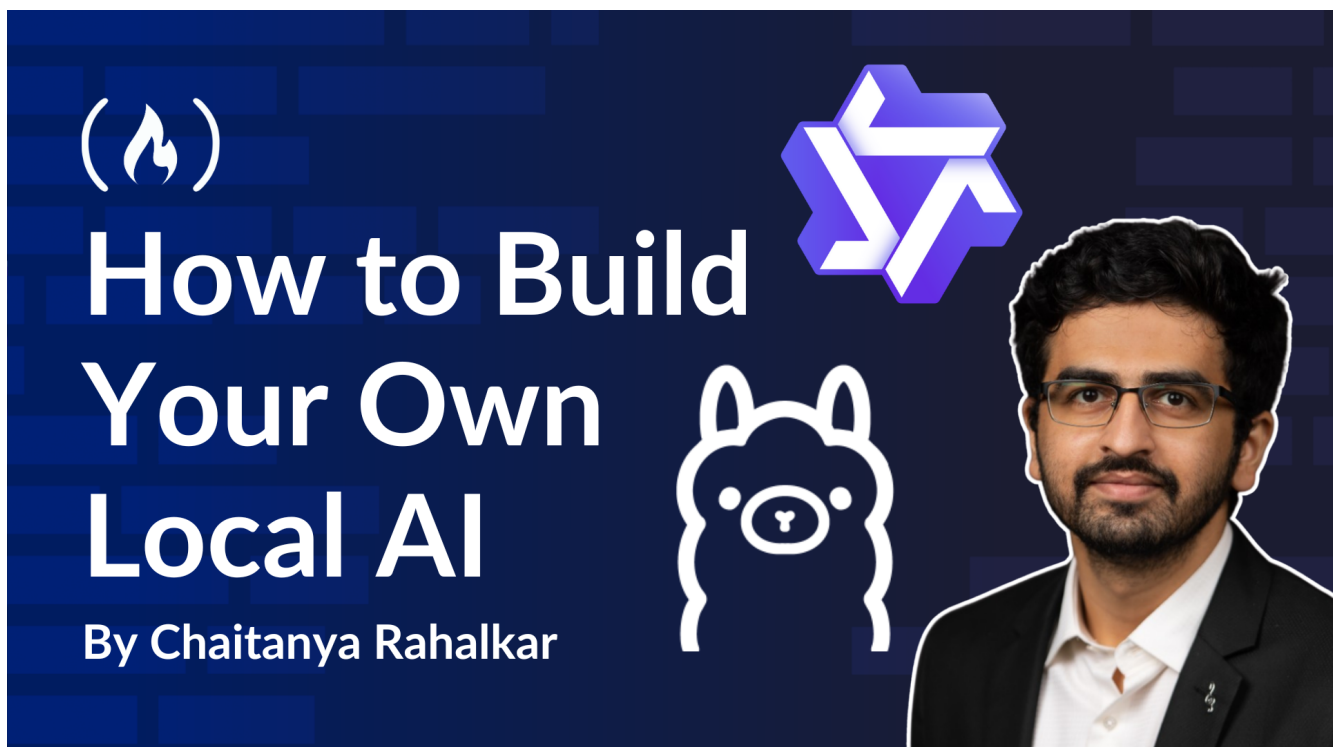


MAY 6, 2025 / #PYTHON

How to Build Your Own Local AI: Create Free RAG and AI Agents with Qwen 3 and Ollama



Chaitanya Rahalkar



The landscape of Artificial Intelligence is rapidly evolving, and one of the most exciting trends is the ability to run powerful Large Language Models (LLMs) directly on your local machine.

This shift away from reliance on cloud-based APIs offers significant advantages in terms of privacy, cost-effectiveness, and offline accessibility. Developers and enthusiasts can now experiment with and deploy sophisticated AI capabilities without sending data externally or incurring API fees.

Ollama, a tool that dramatically simplifies running LLMs locally.

Prerequisites

Before diving into this tutorial, you should have a foundational understanding of Python programming and be comfortable using the command line or terminal. Make sure you have Python 3 installed on your system.

While prior experience with AI or Large Language Models (LLMs) is beneficial, it's not essential, as I'll introduce and explain core concepts like Retrieval-Augmented Generation (RAG) and AI agents throughout the guide.

This tutorial serves as a practical, hands-on guide to harnessing this local AI power. It focuses on leveraging the Qwen 3 family of LLMs, a state-of-the-art open-source offering from Alibaba, combined with Ollama, a tool that dramatically simplifies running LLMs locally.

Table of Contents

1. [Local AI Power with Qwen 3 and Ollama](#)
 - [Ollama: Your Local LLM Gateway](#)
 - [Tutorial Roadmap](#)
2. [How to Set Up Your Local AI Lab](#)
 - [Install Ollama](#)
 - [Choose Your Qwen 3 Model](#)
 - [Pull and Run Qwen 3 with Ollama](#)
 - [Set Up Your Python Environment](#)
3. [How to Build a Local RAG System with Qwen 3](#)
 - [Step 1: Prepare Your Data](#)
 - [Step 2: Load Documents in Python](#)
 - [Step 3: Split Documents](#)

- [Step 6: Index Documents \(Embed and Store\)](#)
- [Step 7: Build the RAG Chain](#)
- [Step 8: Query Your Documents](#)

4. [How to Create Local AI Agents with Qwen 3](#)

- [Step 1: Define Custom Tools](#)
- [Step 2: Set up the Agent LLM](#)
- [Step 3: Create the Agent Prompt](#)
- [Step 4: Build the Agent](#)
- [Step 5: Create the Agent Executor](#)
- [Step 6: Run the Agent](#)

5. [Advanced Considerations and Troubleshooting](#)

- [Controlling Qwen 3's Thinking Mode with Ollama](#)
- [Managing Context Length \(num_ctx\)](#)
- [Hardware Limitations and VRAM](#)

6. [Conclusion and Next Steps](#)

Local AI Power with Qwen 3 and Ollama

Running LLMs locally addresses several key concerns associated with cloud-based AI services.

- Privacy is paramount – data processed locally never leaves the user's machine.
- Cost is another major factor – utilizing open-source models and tools like Ollama eliminates API subscription fees and pay-per-token charges, making advanced AI accessible to everyone.
- Local execution enables offline functionality – crucial for applications where internet connectivity is unreliable or undesirable.

accessible on local hardware. It's a command-line tool that simplifies the download, setup, and execution of various open-source LLMs across macOS, Linux, and Windows.

Ollama handles the complexities of model configuration and GPU utilization, providing a straightforward interface for developers and users. It also exposes an OpenAI-compatible API endpoint, allowing seamless integration with popular frameworks like LangChain.

Tutorial Roadmap

This tutorial will guide you through the process of:

1. **Setting up a local AI environment:** Installing Ollama and selecting/running appropriate Qwen 3 models.
2. **Building a local RAG system:** Creating a system that allows chatting with personal documents using Qwen 3, Ollama, LangChain, and ChromaDB for vector storage.
3. **Creating a basic local AI agent:** Developing a simple agent powered by Qwen 3 that can utilize custom-defined tools (functions).

How to Set Up Your Local AI Lab

The first step is to prepare your local machine with the necessary tools and models.

Install Ollama

Ollama provides the simplest path to running LLMs locally.

- **Linux / macOS:** Open a terminal and run the official installation script:

```
curl -fsSL https://ollama.com/install.sh | sh
```

- **Windows:** Download the installer from the Ollama website (<https://ollama.com/download>) and follow the setup

running:

```
ollama --version
```

Ollama typically stores downloaded models in `~/.ollama/models` on macOS and `/usr/share/ollama/.ollama/models` on Linux/WSL.

Choose Your Qwen 3 Model

Selecting the right Qwen 3 model is crucial and depends on your intended task and available hardware, primarily system RAM and GPU VRAM. Running larger models requires more resources but generally offers better performance and reasoning capabilities.

Qwen 3 offers two main architectures available through Ollama:

- **Dense Models:** (like `qwen3:0.6b`, `qwen3:4b`, `qwen3:8b`, `qwen3:14b`, `qwen3:32b`) These models activate all their parameters during inference. Their performance is predictable, but resource requirements scale directly with parameter count.
- **Mixture-of-Experts (MoE) Models:** (like `qwen3:30b-a3b`) These models contain many "expert" sub-networks but only activate a small fraction for each input token. This allows them to achieve the performance characteristic of their large total parameter count (for example, 30 billion) while having inference costs closer to their smaller *active* parameter count (for example, 3 billion). They offer a compelling balance of capability and efficiency, especially for reasoning and coding tasks.

Recommendation for this tutorial: For the examples that follow, `qwen3:8b` strikes a good balance between capability and resource requirements for many modern machines. If resources are more constrained, `qwen3:4b` is a viable alternative. The MoE model `qwen3:30b-a3b` offers excellent performance, especially for coding

Pull and Run Qwen 3 with Ollama

Once you've chosen a model, you'll need to download it (pull it) via Ollama.

Pull the model: Open the terminal and run (replace `qwen3:8b` with the desired tag):

```
ollama pull qwen3:8b
```

This command downloads the model weights and configuration.

Run interactively (optional test): To chat directly with the model from the command line:

```
ollama run qwen3:8b
```

Type prompts directly into the terminal. Use `/bye` to exit the session. Other useful commands within the interactive session include `/?` for help and `/set parameter <name> <value>` (for example, `/set parameter num_ctx 8192`) to temporarily change model parameters for the current session. Use `ollama list` outside the session to see downloaded models.

Run as a server: For integration with Python scripts (using LangChain), Ollama needs to run as a background server process, exposing an API. Open a *separate* terminal window and run:

```
ollama serve
```

Keep this terminal window open while running the Python scripts. This command starts the server, typically listening on `http://`

Set Up Your Python Environment

A dedicated Python environment is recommended for managing dependencies.

Create a virtual environment:

```
python -m venv venv
```

Activate the environment:

- macOS/Linux: `source venv/bin/activate`
- Windows: `venv\Scripts\activate`

Install necessary libraries:

```
pip install langchain langchain-community langchain-core langchain-ollama chromadb sentence-transformers pypdf python-dotenv unstructured[pdf] tiktoken
```

- `langchain`, `langchain-community`, `langchain-core`: The core LangChain framework for building LLM applications.
- `langchain-ollama`: Specific integration for using Ollama models with LangChain.
- `chromadb`: The local vector database for storing document embeddings.
- `sentence-transformers`: Used for an alternative local embedding method (explained later).
- `pypdf`: A library for loading PDF documents.
- `python-dotenv`: For managing environment variables (optional but good practice).
- `unstructured[pdf]`: An alternative, powerful document loader, especially for complex PDFs.
- `tiktoken`: Used by LangChain for token counting.

ChromaDB. Ensuring compatibility between these pieces and correctly configuring parameters (like Ollama's context window size or selecting a model appropriate for the available VRAM) is key to a smooth experience.

While this modularity offers flexibility – allowing components like the LLM or vector store to be swapped – it also means the initial setup requires careful attention to detail. This tutorial aims to provide clear steps and sensible defaults to minimize potential friction points.

How to Build a Local RAG System with Qwen 3

Retrieval-Augmented Generation (RAG) is a powerful technique that enhances LLMs by providing them with external knowledge.

Instead of relying solely on its training data, the LLM can retrieve relevant information from a specified document set (like local PDFs) and uses that information to answer questions. This significantly reduces "hallucinations" (incorrect or fabricated information) and allows the LLM to answer questions about specific, private data without needing retraining.

The core RAG process involves:

1. Loading and splitting documents into manageable chunks.
2. Converting these chunks into numerical representations (embeddings) using an embedding model.
3. Storing these embeddings in a vector database for efficient searching.
4. When a query comes in, embedding the query and searching the vector database for the most similar document chunks.
5. Providing these relevant chunks (context) along with the original query to the LLM to generate an informed answer.

Step 1: Prepare Your Data

Create a directory named `data` in the project folder. Place the PDF document that you intend to query into this directory. For this tutorial, using a single, primarily text-based PDF (like a research paper or a report) for simplicity.

```
mkdir data
# Copy your PDF file into the 'data' directory
# e.g., cp ~/Downloads/some_paper.pdf./data/mydocument.pdf
```

If you don't have a PDF readily available that you'd like to use, you can download a sample PDF (the Llama 2 paper) for this tutorial using the following command in your terminal:

```
wget --user-agent "Mozilla" "https://arxiv.org/pdf/2307.09288.pdf" -O
```

This command creates the `data` directory and downloads the PDF, saving it as `llama2.pdf` inside the `data` directory. If you prefer to use your own document, place your PDF file into the `data` directory and update the filename in the subsequent Python code.

Step 2: Load Documents in Python

Use LangChain's document loaders to read the PDF content.

`PyPDFLoader` is straightforward for simple PDFs.

`UnstructuredPDFLoader` (requires `unstructured[pdf]`) can handle more complex layouts but has more dependencies.

```
# rag_local.py
import os
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader # Or Uns

load_dotenv() # Optional: Loads environment variables from .env file
```

```
def load_documents():
    """Loads documents from the specified data path."""
    pdf_path = os.path.join(DATA_PATH, PDF_FILENAME)
    loader = PyPDFLoader(pdf_path)
    # loader = UnstructuredPDFLoader(pdf_path) # Alternative
    documents = loader.load()
    print(f"Loaded {len(documents)} page(s) from {pdf_path}")
    return documents

# documents = load_documents() # Call this later
```

Step 3: Split Documents

Large documents need to be split into smaller chunks suitable for embedding and retrieval. The `RecursiveCharacterTextSplitter` attempts to split text semantically (at paragraphs, sentences, and so on) before resorting to fixed-size splits. `chunk_size` determines the maximum size of each chunk (in characters), and `chunk_overlap` specifies how many characters should overlap between consecutive chunks to maintain context.

```
# rag_local.py (continued)
from langchain_text_splitters import RecursiveCharacterTextSplitter

def split_documents(documents):
    """Splits documents into smaller chunks."""
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200,
        length_function=len,
        is_separator_regex=False,
    )
    all_splits = text_splitter.split_documents(documents)
    print(f"Split into {len(all_splits)} chunks")
    return all_splits

# loaded_docs = load_documents()
# chunks = split_documents(loaded_docs) # Call this later
```

Step 4: Choose and Configure Embedding Model

Embeddings transform text into vectors (lists of numbers) such that